

AGENCYOS

PHASE 4 - QA / PROTOTYPE VALIDATION AGENT

Task 2: Full UI Design + UI Prototype

RESPONSIBILITIES + VALIDATION MATRIX + DEFECT / RETEST LOOP + IMPLEMENTATION + TESTING SPECIFICATION

Official Role	QA / Prototype Validation Agent
Phase	Phase 4 - Task 2
Primary Mission	Independently validate the exact prototype build against the locked approved UI, approved scope and critical review flows before Admin and Client review
Primary Input	Exact PrototypeBuild + source UI version + coverage matrix + approved screens/flows + known limitations
Primary Output	Prototype QA report + defects/evidence + PASS / CHANGES_REQUIRED decision for the exact build
Review Order	Prototype Agent → QA / Prototype Validation → Admin → PM → Client
Hard Rule	Prototype Agent saying 'fixed' or build succeeding does not close a defect; QA must retest and verify
Boundary	This Phase 4 validation is not the Phase 6 Master QA / production-readiness certification

This specification focuses on Phase 4 prototype validation. It preserves the locked review order, exact build/version traceability, defect evidence, fix/retest loops, Admin handoff, revision regression and security boundaries.

1. Agent Mission & Scope

The QA / Prototype Validation Agent independently proves that the prototype build the Admin and Client are about to review actually represents the exact approved UI baseline and works as a reviewable product experience. It checks critical screens, routes, buttons, interactions, states, layout integrity, scope coverage, source-version linkage, build integrity and documented limitations.

This is an independent validation role. It does not create the prototype, approve the prototype on behalf of Admin/Client, verify payments, or certify production readiness for launch.

2. Distinction From Other QA Roles

QA role	Purpose	Phase 4 Prototype Validation Agent relationship
Design QA	Validates high-fidelity UI before client UI approval	Consumes approved Design QA outcome as input; does not repeat theme selection
Prototype Validation QA	Validates interactive prototype/build before Admin/client prototype review	This document
Phase 5 development QA	Validates functional product during development	Later lifecycle; deeper backend/API/business behavior
Phase 6 Master QA	360° final QA / production readiness	Separate final certification; must not be replaced by Phase 4 prototype validation

Phase 4 Prototype Validation is intentionally narrower than final production QA, but it must still be strict enough that Admin/Client do not waste review cycles on obviously broken or mismatched builds.

3. Start / Intake Conditions

- ☐ PrototypeBuild exists and has a unique build_id/version.
- ☐ PrototypeBuild points to exact source_ui_version_id.
- ☐ Source UI version is the client-approved locked Phase 4 UI baseline.
- ☐ Prototype coverage list exists.
- ☐ Critical screens/routes/flows to validate are identified.
- ☐ Artifact (URL/APK/package/preview) is accessible to QA.
- ☐ Required test data/credentials are available and safe.
- ☐ Known prototype limitations are declared.
- ☐ Target platforms/viewports relevant to this prototype are known.

If build identity, source UI version, artifact access or critical coverage is missing, QA must reject intake rather than inventing context.

4. Required QA Intake Package

- ☐ project_id and organization_id.
- ☐ build_id / build version.
- ☐ source_ui_version_id.
- ☐ source design/Figma file/page/node/version refs where available.
- ☐ approved screen inventory.
- ☐ Requirement → Feature → Screen → State → Prototype coverage matrix.
- ☐ prototype route/interaction coverage.
- ☐ artifact URL/APK/package.
- ☐ target platform/device/viewports.
- ☐ mock/test data instructions.
- ☐ known limitations and excluded flows.
- ☐ Prototype Agent self-check/build result.
- ☐ revision origin when validating a revised build.

5. Prototype Validation Coverage Model

The validator must test what the prototype claims to demonstrate. It should not turn a Phase 4 review prototype into an accidental Phase 6 production-certification exercise, but every critical review flow must be real and coherent.

Validation dimension	Required checks
Build identity	Exact build/version; correct project/tenant; artifact accessible
Source linkage	Build matches exact approved/locked UI version
Screen coverage	All critical prototype-required screens load
Navigation	Critical routes, back/cancel, tabs, drawers and transitions
Interactions	Expected buttons/CTAs/forms/dialogs behave as specified
States	Important loading, empty, error, success, validation, permission/offline states where applicable
Visual fidelity	Approved structure/theme/tokens/components reasonably match source UI
Layout integrity	No major overflow, clipping, unreadable content or broken responsive layout on target review sizes
Role behavior	Role-specific screens/controls behave according to approved prototype scope
Mock/test data	Realistic review data; no production secrets/live sensitive data
Limitations	Declared limitation is accurate and not hiding a critical broken flow
Security basics	No exposed secrets, obvious cross-project data, unsafe review access

6. What Counts as Prototype QA PASS

- ☐ Exact build identity and source UI version are verified.
- ☐ All critical prototype screens load.
- ☐ All critical routes/flows work.
- ☐ No unresolved defect blocks intended client review.
- ☐ No major visual overflow/layout break exists on required review targets.
- ☐ Required important UI states exist.
- ☐ Approved visual direction is materially consistent with locked UI.
- ☐ Prototype coverage matches the approved scope/coverage matrix for Phase 4.
- ☐ Known limitations are truthful and non-blocking for intended review.
- ☐ No production secrets or cross-project/tenant data are exposed.
- ☐ All QA-found defects required for this build have been fixed and QA-retested

QA PASS belongs to the exact tested build. A later build is not automatically QA-passed.

7. Defect Classification & Severity

Severity	Definition in Phase 4 prototype context	Typical disposition
P0 / Critical	Security/tenant leak, unusable artifact, catastrophic corruption, critical review impossible	BLOCK; immediate fix; no Admin share
P1 / Major	Critical flow broken, key screen unusable, major source-UI mismatch, severe navigation/layout failure	BLOCK; fix + QA retest
P2 / Moderate	Important but non-critical interaction/state/layout issue	Fix before review unless explicitly accepted under policy
P3 / Minor	Cosmetic/non-blocking issue with low review impact	May be fixed or documented according to policy
Enhancement / Scope	New behavior not required by approved prototype	Not a QA defect; route to scope/revision classification

Critical defects cannot be hidden by an average QA score. Hard blockers remain hard blockers.

8. Defect Record Requirements

- ☐ Defect ID.
- ☐ project_id / organization_id.
- ☐ exact build_id/version.
- ☐ source_ui_version_id.
- ☐ screen / route / requirement / flow reference.
- ☐ environment/platform/device/viewport.
- ☐ severity and priority.
- ☐ clear title and reproduction steps.
- ☐ expected result.
- ☐ actual result.
- ☐ evidence: screenshot/video/log where useful.
- ☐ assigned owner.
- ☐ fix build/version reference.
- ☐ QA retest result.
- ☐ regression result where applicable.
- ☐ status history and timestamps.

9. Defect Lifecycle

Possible controlled states: REOPENED, BLOCKED, DUPLICATE, WON'T_FIX, DEFERRED - but only with explicit evidence/reason and correct approval authority.

Developer/Prototype Agent marking FIX_READY is not verification. The QA / Prototype Validation Agent must retest the exact fix build before VERIFIED/CLOSED.

10. Exact Build & Version Integrity Rules

- ☐ QA report stores exact build_id/version.
- ☐ QA report stores exact source_ui_version_id.
- ☐ Evidence links to exact build under test.
- ☐ Fix creates/references a new or revised build identity when build contents changed.
- ☐ QA does not transfer PASS from old build to new build without retest.
- ☐ Admin receives only the exact QA-passed build.
- ☐ PM/client share must reference that same Admin-approved build.
- ☐ If Admin requests an edit, the resulting new build must return to QA before Admin re-review.
- ☐ If Client requests an allowed correction, the new build must return to QA before Admin/PM/client re-share.

11. Validation Test Categories

11.1 Build Smoke

- ☐ Artifact installs/opens/loads as applicable.
- ☐ Initial screen renders.
- ☐ No immediate crash/fatal error.
- ☐ Required test account/data can be accessed.

11.2 Screen Availability

- ☐ All critical screens in prototype coverage exist.
- ☐ No accidental blank/placeholder screen for a required flow.
- ☐ Correct project/brand/content shell is shown.

11.3 Navigation

- ☐ Critical route transitions work.
- ☐ Back/cancel/close behavior does not trap the reviewer.
- ☐ Tabs/drawers/modals open and close correctly.
- ☐ No broken loop/dead end in the primary journey unless intentionally documented.

11.4 Interactions

- ☐ Buttons/CTAs expected to work are functional.
- ☐ Forms/controls accept representative input.
- ☐ Validation appears where required.
- ☐ Success/failure response is visible where important.

11.5 Required States

- ☐ Loading states where important.
- ☐ Empty states.
- ☐ Validation errors.
- ☐ System/error states where prototype scope includes them.
- ☐ Success states.
- ☐ Permission/restricted/offline state where relevant.

11.6 Visual Consistency

- ☐ Locked theme/colors and component intent are applied.
- ☐ Typography/spacing/hierarchy do not materially diverge from approved UI.
- ☐ Assets/icons/images match approved source or documented replacement.

11.7 Layout / Responsive

- ☐ No major clipping/overflow.
- ☐ Long text and representative extreme values do not destroy layout.
- ☐ Required review viewports/platforms remain usable.

11.8 Data / Privacy

- ☐ Mock/test data is clearly non-production.
- ☐ No secrets/tokens/production credentials exposed.
- ☐ No other client's/project's data visible.

12. Prototype Regression Rules

- ☐ Every revised build is checked against the changed area.
- ☐ Critical primary flows receive regression after a fix that can affect routing/shared components/state.
- ☐ Previously fixed critical defects must not reappear.
- ☐ Admin-origin edits trigger regression of affected flows before Admin re-review.
- ☐ Client-origin revisions trigger regression of affected flows before Admin/client re-share.
- ☐ Regression scope should be risk-based, not blindly re-run every possible check when unnecessary.

13. Scope vs Defect Decision Rules

Observed issue	Classification
Prototype differs from approved screen/flow	Defect
Approved button does nothing but should be interactive	Defect
Client now wants a new feature not in approved scope	Scope/change request, not defect
Client dislikes a valid approved visual choice	Revision/design preference, not necessarily defect
Known documented prototype limitation matches approved review scope	Not a defect unless it blocks intended review
Prototype hides a required feature behind 'limitation'	Defect / scope-coverage failure

14. QA Decision Outputs

- ☐ QA_PASS - exact build may proceed to Admin review.
- ☐ CHANGES_REQUIRED - build returns to Prototype Agent with structured defects.
- ☐ BLOCKED_EXTERNAL - validation cannot complete because external dependency/artifact is unavailable.
- ☐ INVALID_INTAKE - build/source/coverage package is incomplete or wrong

QA must not output ADMIN_APPROVED or CLIENT_APPROVED. Those are human/workflow decisions outside this agent's authority.

15. Admin Handoff Package

- ☐ Exact build ID/version.
- ☐ Exact source UI version.
- ☐ Artifact URL/APK/package.
- ☐ Prototype coverage summary.
- ☐ QA result.
- ☐ Open defects by severity, if any are allowed by policy.
- ☐ Known limitations.
- ☐ Platforms/viewports tested.
- ☐ Defect/fix/retest evidence.
- ☐ Regression result.
- ☐ Date/time and QA run identity.

Only a QA-passed exact build should reach Admin prototype review under the locked Phase 4 flow.

16. Admin Edit & Client Revision QA Loops

16.1 Admin EDIT

ADMIN EDIT → Prototype Agent creates revised build → QA validates exact revised build → QA PASS → Admin re-review

16.2 Client Allowed Revision

CLIENT FEEDBACK → PM revision request → Prototype Agent revised build → QA validates exact revised build → Admin review → PM re-share

16.3 Client Scope Change

CLIENT NEW FUNCTIONALITY → PM/Scope classification → Change Request workflow; QA does not reinterpret it as a prototype bug

17. QA Agent Must NOT

- ☐ Must not edit production/business scope.
- ☐ Must not implement the prototype itself except dedicated test harness/fixtures where allowed.
- ☐ Must not mark Admin/client approval.
- ☐ Must not verify payment.
- ☐ Must not certify Phase 6 production readiness from a Phase 4 prototype review.
- ☐ Must not hide critical defects using an average score.
- ☐ Must not test a different build than the one later shared to Admin/client.
- ☐ Must not close defects merely because Prototype Agent says 'fixed'.
- ☐ Must not invent missing requirements or UI behavior.

18. Admin Panel - QA / Validation Visibility

Admin surface	Must show
Prototype QA Overview	build, source UI version, QA state, owner, blocker, latest run
Validation Matrix	screens/routes/interactions/states tested and result
Defects	severity, build, screen/flow, evidence, status, owner
Retests	fix build, retest result, evidence
Regression	scope, results and any reopened defects
Build History	which builds passed/failed and why
Known Limitations	declared prototype exclusions/limits
Admin Review Readiness	explicit QA PASS / CHANGES_REQUIRED / BLOCKED state
Activity/Audit	QA runs, defects, fixes, retests, state transitions

Admin should never need to infer 'probably tested' from a build status. QA evidence and exact-build linkage must be visible.

19. Data Contracts

Entity	Minimum fields
PrototypeQARun	id, project/organization, build_id, source_ui_version_id, environment, started_at, completed_at, result, coverage summary
PrototypeQACheck	qa_run_id, category, target screen/route/state, expected, actual, result, evidence refs
PrototypeDefect	id, build_id, requirement/screen/route, severity, repro, expected/actual, evidence, status, assignee
PrototypeRetest	defect_id, fix_build_id, retest result, evidence, regression result
PrototypeQAEvidence	qa_run/check/defect ref, artifact type, storage ref, metadata
PrototypeQABlocker	build_id, reason, owner, dependency, resume condition
AuditEvent	actor/agent, action, build/defect, before/after, timestamp, correlation

20. Event & Job Contracts

Event / job	QA behavior
PrototypeBuildReady	Create/queue validation for exact build
PrototypeQAShared	Persist run identity and exact build/source UI refs
PrototypeQAChangesRequired	Publish structured defect outcome to Prototype Agent
PrototypeFixReady	Queue defect retest against fix build
PrototypeQAPassed	Make exact build eligible for Admin review
PrototypeQABlocked	Expose blocker and resume condition
AdminPrototypeEditRequested	After revised build exists, run QA again
ClientPrototypeChangeRequested	After PM/Prototype Agent produce revised build, run QA again

- ☐ Duplicate PrototypeBuildReady event does not create conflicting QA decisions.
- ☐ QA run is tied to one exact build.
- ☐ Retest job is tied to defect + fix build.
- ☐ Worker failure is resumable from persisted state.
- ☐ Failed QA job is visible in operations/Admin.

21. Security & Permission Boundaries

- ☐ QA can read only authorized project/build/test artifacts.
- ☐ QA cannot access another tenant/project's prototype data.
- ☐ QA evidence must not leak credentials/tokens.
- ☐ QA may create/update QA records and defects but cannot perform Admin/client approvals.
- ☐ QA may not verify money or invoke production release.
- ☐ Review links/credentials used for validation are stored/handled securely.
- ☐ Privileged QA state transitions are audited.

22. Cost & Efficiency Rules

- ☐ Use deterministic automated checks where possible for route/build/state validation.
- ☐ Use AI reasoning only where semantic/visual/requirement comparison benefits from it.
- ☐ Do not rerun the full matrix unnecessarily after a tiny isolated change; apply risk-based regression.
- ☐ Cache/reuse immutable approved UI references and coverage metadata.
- ☐ Avoid duplicate screenshots/video/evidence capture when existing evidence is valid for the same exact build.
- ☐ Track QA agent/model/tool usage where AgencyOS telemetry exists.

23. Failure & Recovery

Failure	Required behavior
Artifact cannot open/install	QA BLOCKED/FAIL; no Admin review
Build crashes	P0/P1 defect according to impact
Test credentials invalid	BLOCKED_EXTERNAL or test-data blocker; do not fake result
Source UI reference unavailable	INVALID_INTAKE/BLOCKED; no fidelity claim
QA worker crashes	Persist run; safe retry/resume
Evidence upload fails	QA decision should not lose defect data; retry evidence storage
Duplicate event	Idempotent QA run/business result
Fix build not linked to defect	Retest rejected until exact linkage exists
Admin/client revision arrives	Validate new exact build; do not reuse old PASS

24. Technical Implementation Plan

QAP4-I01 Existing QA Discovery

Inspect current QA Agent, PrototypeBuild domain, defect model, Admin review flow, events/jobs, test harness and evidence storage.

QAP4-I02 QA Contract

Define intake/output schemas, PASS/CHANGES_REQUIRED/BLOCKED/INVALID_INTAKE outcomes and permission boundaries.

QAP4-I03 Exact Build Guard

Enforce build_id + source_ui_version_id integrity and project/tenant scope.

QAP4-I04 Validation Matrix Engine

Implement screen/route/interaction/state/visual/build/security check tracking.

QAP4-I05 QA Run Domain

Create/extend PrototypeQARun and per-check evidence/result storage.

QAP4-I06 Defect Domain

Create/extend prototype defect severity/status/evidence/build linkage.

QAP4-I07 Retest Workflow

Implement FIX_READY → QA_RETEST → VERIFIED/CLOSED with exact fix build.

QAP4-I08 Regression Workflow

Implement risk-based regression and reopened-defect handling.

QAP4-I09 Admin Handoff

Expose only exact QA-passed build with QA package.

QAP4-I10 Revision Revalidation

Force revised Admin/client builds back through QA.

QAP4-I11 Admin Panel

Expose matrix, defects, retests, regression, blockers and exact-build QA status.

QAP4-I12 Reliability

Idempotent jobs/events, worker recovery and evidence-upload retry.

QAP4-I13 Security

Tenant/project isolation, secure evidence, no approval/payment privilege.

QAP4-I14 Automated Test Harness

Add deterministic route/state/build checks where project tooling supports them.

QAP4-I15 E2E & Regression

Verify full Prototype Agent→QA→Admin→PM/client loop and earlier shared Phase 4 behavior.

25. QA / Prototype Validation Test Matrix

Area	Required coverage
Intake	valid build, missing build, wrong project, unapproved source UI, inaccessible artifact
Build smoke	launch/install/open, initial route, crash/fatal error
Coverage	critical screen/route/flow mapping
Navigation	routes, back/cancel, tabs, drawers, dead ends
Interactions	buttons, CTAs, forms, validation, dialogs
States	loading, empty, error, success, permission/offline where applicable
Visual fidelity	approved source UI/theme/components
Layout	overflow, clipping, long content, required responsive targets
Data/security	mock data, no secrets, no cross-project/tenant leak
Defects	severity, evidence, exact build linkage, status lifecycle
Retest	fix build link, verified result, defect closure
Regression	affected critical flows and escaped defect prevention
Admin gate	QA PASS required; exact build only
Revision	Admin/client revision invalidates previous PASS for new build
Reliability	duplicate event, worker crash, evidence failure

26. Mandatory Detailed Test Cases

ID	Scenario	Expected result
QAP4-T001	Valid PrototypeBuild intake	QA run starts with exact build/source refs
QAP4-T002	Build from unapproved UI	Intake rejected/blocked
QAP4-T003	Build belongs to another project	Access denied
QAP4-T004	Artifact URL/APK unavailable	QA blocked; no Admin handoff
QAP4-T005	Prototype crashes on launch	Critical defect; QA FAIL
QAP4-T006	Critical screen missing	Defect/coverage FAIL
QAP4-T007	Critical route broken	P1 defect; CHANGES_REQUIRED
QAP4-T008	Back action traps reviewer	Defect
QAP4-T009	Required CTA is decorative/nonfunctional	Defect
QAP4-T010	Required form validation absent	Defect
QAP4-T011	Required error state absent	Defect
QAP4-T012	Required empty state absent	Defect
QAP4-T013	Major layout overflow on target viewport	Defect
QAP4-T014	Visual theme materially mismatches locked UI	Defect
QAP4-T015	Prototype build/source UI mismatch	QA FAIL
QAP4-T016	Production secret exposed in build	P0 security defect
QAP4-T017	Another client's data appears	P0 security defect
QAP4-T018	Prototype Agent marks defect fixed	Defect remains FIX_READY until QA retest
QAP4-T019	Retest on wrong build	Retest rejected
QAP4-T020	Retest exact fix build succeeds	Defect VERIFIED/CLOSED
QAP4-T021	Fix breaks adjacent critical route	Regression FAIL; reopen/block
QAP4-T022	QA PASS exact build	Admin review becomes eligible
QAP4-T023	Later new build created	Old PASS not inherited

ID	Scenario	Expected result
QAP4-T024	Admin EDIT creates revised build	New QA run required
QAP4-T025	Client allowed revision creates new build	New QA run required before re-share
QAP4-T026	Client new feature request	Not misclassified as QA defect
QAP4-T027	Duplicate PrototypeBuildReady event	No conflicting duplicate QA outcome
QAP4-T028	QA worker crashes	Run safely resumes/retries
QAP4-T029	Evidence upload fails	Defect/result preserved; evidence retryable
QAP4-T030	QA agent tries Admin approval	Denied
QAP4-T031	QA agent tries payment verification	Denied
QAP4-T032	Cross-tenant QA evidence access	Denied

27. Mandatory End-to-End Validation Scenario

- ☐ Receive a PrototypeBuild produced from exact client-approved locked UI.
- ☐ Validate intake package and exact source linkage.
- ☐ Run smoke, critical screens, routes, interactions, states, visual/layout and security-basic checks.
- ☐ Find one broken route and one major overflow.
- ☐ Create defects with exact build/evidence/severity.
- ☐ Return CHANGES_REQUIRED.
- ☐ Prototype Agent creates revised build and marks fixes ready.
- ☐ QA retests exact fix build.
- ☐ Verify both defects; run regression on affected primary flow.
- ☐ Return QA_PASS for exact revised build.
- ☐ Admin reviews and requests one edit.
- ☐ New build is created; previous QA PASS does not transfer.
- ☐ QA validates revised Admin-edit build and passes it.
- ☐ PM/client review happens.
- ☐ Client requests one allowed correction.
- ☐ Another revised build returns through QA.
- ☐ QA passes exact final revised build.
- ☐ Admin approves exact QA-passed build.
- ☐ PM shares same exact build to client.
- ☐ Final client approval references that build.
- ☐ All QA runs, defects, retests and revisions remain visible in Admin history.

28. Definition of Done - QA / Prototype Validation Agent

- ☐ Agent accepts only valid exact build intake.
- ☐ Exact source UI version is verified.
- ☐ Critical screen/route/interaction/state coverage is testable and recorded.
- ☐ Visual/layout validation checks approved prototype review targets.
- ☐ Mock/test data and security basics are validated.
- ☐ Defect severity/status/evidence are structured.
- ☐ Prototype Agent cannot self-close defects.
- ☐ Fix/retest loop uses exact fix build.
- ☐ Regression is applied according to risk.
- ☐ Only exact QA-passed build is eligible for Admin review.
- ☐ Admin/client revisions force revalidation of the new build.
- ☐ Admin Panel exposes QA runs, defects, retests, blockers and build lineage.
- ☐ Events/jobs are idempotent and recoverable.
- ☐ Project/tenant isolation and permission boundaries pass.
- ☐ No unresolved Phase 4 Prototype Validation P0/P1 defects remain.

29. Master Checklist

A. Intake

- ☐ build ID
- ☐ source UI version
- ☐ coverage
- ☐ artifact
- ☐ test data
- ☐ targets
- ☐ limitations

B. Smoke

- ☐ install/open
- ☐ initial route
- ☐ fatal error/crash
- ☐ test account/data

C. Screens & Routes

- ☐ critical screens
- ☐ routes
- ☐ back/cancel
- ☐ tabs/drawers
- ☐ dead ends

D. Interactions

- ☐ buttons
- ☐ CTAs
- ☐ forms
- ☐ validation
- ☐ dialogs
- ☐ success/failure behavior

E. States

- ☐ loading
- ☐ empty
- ☐ error
- ☐ success
- ☐ permission/offline where relevant

F. Visual/Layout

- ☐ theme fidelity
- ☐ components
- ☐ spacing/typography
- ☐ overflow
- ☐ long content
- ☐ responsive targets

G. Data/Security

- ☐ mock data
- ☐ no production secrets
- ☐ no cross-project/tenant data
- ☐ safe artifact access

H. Defects

- ☐ severity
- ☐ evidence
- ☐ build link
- ☐ owner
- ☐ status
- ☐ fix build

I. Retest

- ☐ exact fix build
- ☐ verified result
- ☐ regression

- ☐ close/reopen

J. Admin Gate

- ☐ QA PASS
- ☐ exact build
- ☐ coverage summary
- ☐ defects/limitations
- ☐ handoff

K. Revision QA

- ☐ Admin-edit build revalidated
- ☐ client-revision build revalidated
- ☐ old PASS not inherited

L. Reliability

- ☐ idempotency
- ☐ worker retry
- ☐ evidence retry
- ☐ blocker visibility
- ☐ audit

M. Permissions

- ☐ no Admin approval
- ☐ no client approval
- ☐ no payment verification
- ☐ tenant/project scope

N. Testing

- ☐ unit
- ☐ integration
- ☐ event/job
- ☐ security
- ☐ negative
- ☐ recovery
- ☐ E2E
- ☐ regression

30. Final Locked QA / Prototype Validation Flow

PROTOTYPE BUILD READY → QA INTAKE → VERIFY BUILD + SOURCE UI → SMOKE →
SCREEN/ROUTE/INTERACTION/STATE VALIDATION → VISUAL/LAYOUT/SECURITY BASIC CHECKS → DEFECTS? →
CHANGES_REQUIRED → PROTOTYPE AGENT FIX → FIX BUILD → QA RETEST → REGRESSION → QA PASS →
ADMIN REVIEW → ADMIN EDIT? → NEW BUILD → QA AGAIN → ADMIN APPROVED → PM/CLIENT REVIEW →
CLIENT REVISION? → NEW BUILD → QA AGAIN → FINAL QA PASS → ADMIN → PM → CLIENT FINAL

Locked validation principle: only the exact build that was independently tested and passed may proceed to Admin review. Any changed build must be revalidated; 'fixed' is not 'verified'.